Drone Simulator Documentation

1. Introduction & Project Overview

The Drone Simulator project is an interactive, real-time web application featuring a dual-device architecture. It enables a high-performance 3D flight rendering canvas on a primary web browser (the **Viewer**) while delegating manual flight telemetry inputs and look-angles to a secondary mobile interface (the **Controller**).
The underlying framework utilizes **Three.js** for graphics and physics calculations alongside a **Node.js** and **Socket.io** server infrastructure to handle synchronous coordination between connected network nodes. It features collisions handled by localized octrees, responsive camera mechanics supporting third-person (TPV) and first-person (FPV) layouts, adaptive battery and range tracking, and automated network host resolution for painless local network cross-play.

Component	Technology Stack	Primary Responsibility
3D Simulation Engine	Three.js / Vue 3 (Vite)	Renders environments, computes drone aerodynamics, assesses bounding collision hulls, and displays the workspace UI.
Mobile Controller HUD	HTML5 Touch API / Vue 3	Translates dual-joystick position adjustments, tracks gimbal camera profiles, and visualizes device telemetry.
Network Relay Broker	Node.js / Socket.io	Bridges WebSocket traffic, aggregates virtual rooms, filters validation credentials, and ensures low-latency control streams.

2. Network Automation & Architectural Optimization

To eliminate manual configurations during network initialization, the application dynamically evaluates structural hosting environments on boot. Local system interfaces are processed using the platform native os.networkInterfaces() abstraction layer to capture active LAN adapters

while filtering out loopback boundaries (127.0.0.1 / localhost).

- **Vite Configuration:** The frontend bundler binds its engine to 0.0.0.0, forcing the application to listen across the entire Local Area Network. On startup, Vite uses the discovered system IP adapter to open the browser page natively on the explicit IP string, replacing arbitrary local address tags.
- **Vue Frontend Capture:** The client reads the initialized page window metrics via window.location.hostname. This instantly locks down the target socket destination parameter without code modifications, keeping the client runtime and backend relay securely aligned.
- **Automated QR Synchronization:** The Viewer interface generates an integrated QR code pointing directly to the mobile controller route using the verified LAN IP, allowing immediate, plug-and-play synchronization for hand-held mobile remote devices.

3. Mechanics, Simulation, and Systems Logic

A. Multi-Touch Flight Sticks

The control grid leverages dual non-static thumb-stick controls backed by PointerEvent tracking for seamless cross-browser touch handling. Input normalization converts pixel offsets into clean vector bounds scaled against a predefined radius of 55 pixels. Dead-zone clamping suppresses subtle thumb adjustments below 20% to prevent drift, ensuring a crisp and intentional piloting experience.

B. Flight Kinematics and Movement Vector Calculations

Movement calculations map joystick deflections across decoupled world coordinates.
Right-hand controls influence directional planar vectors along the ground relative to the current drone alignment, while left-hand control paths handle ascending altitude adjustments and rotational yaw values around the vertical axis. The simulation aggregates these vectors dynamically per frame using a frame-rate independent delta scaling approach, applying an exponential damping friction factor of 0.9 to emulate aerodynamic drag.

C. Advanced Camera Configurations (TPV & FPV)

The display grid features two distinct visualization profiles:

1. **Third-Person View (TPV):** Features a smooth trailing camera array positioned behind the drone using spherical linear interpolations (Slerp), creating an organic follow effect.
2. **First-Person View (FPV):** Mounts the camera directly inside the drone assembly node graph. It uses a dynamic gimbal tilt pivot that maps range slider modifications to camera rotation vectors up to a maximum physical ceiling of 35 degrees. Zoom inputs dynamically adjust the field-of-view (FOV) projection matrix to replicate actual focal length alterations.

D. Environment Collision Assessment

Static scenery and mesh models are parsed directly into space-partitioned **Octree structures** upon loading. During each frame loop, the drone's collision volume is treated as a 3D mathematical sphere. If an intersection is caught by the spatial octree partition, the simulator

calculates the collision depth vector, instantly projects the drone outwards along the collision normal, handles velocity dampening, and triggers real-time sound effects based on impact force.

E. Simulated Telemetry (Battery & Signal Strength)

Telemetry metrics update continuously to provide realistic operational constraints during flight:

- **Linear Power Depletion:** Battery drainage scales down linearly against an established 5-minute flight session window, transitioning from 100% to 0% before sending power states down the telemetry socket pipeline.
- **Proximity Signal Scaling:** Signal tracking computes Euclidean distance offsets between the drone's position and the starting home base. Signal strength stays full within a 30-meter radius, degrades linearly beyond that point, and drops to zero if the drone passes outside the maximum 180-meter communication range.

4. Operational Instructions

To run the application, execute the **OPEN THIS FILE TO RUN GAME.bat** file. This batch launcher handles starting both the Node.js WebSocket signaling server and the Vite frontend application. Once started, your system browser will automatically open to the live network IP address, displaying the simulation environment and showing the connection QR code for your mobile controller.